

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	RAGAVIS	Reg. No.:	192565027
Section / Batch:	SLOT – A (2025-29)	Date:	27-08-2026

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time embedded system receives interrupt requests from sensors, communication modules, and storage devices at unpredictable intervals. Develop pseudocode for a nested interrupt handling mechanism that allows a higher-priority interrupt to pre-empt a lower-priority service routine. The algorithm should save the current processor context, service the higher-priority interrupt, restore the interrupted routine, and ensure that pending interrupts are handled correctly.
2. A data-intensive embedded system must transfer a large amount of information between memory and multiple I/O devices without continuously occupying the processor. Write pseudocode for a scatter-gather DMA mechanism that transfers data stored in multiple non-contiguous memory locations. The algorithm should maintain a list of memory descriptors, initiate successive transfers, update descriptor status, and handle completion and error conditions.
3. A computer system supports several I/O devices that generate data at different rates, causing temporary variations in the amount of data waiting for processing. Design pseudocode for an I/O buffering mechanism that manages incoming and outgoing data using appropriate buffer allocation and release strategies. The algorithm should monitor buffer occupancy, handle buffer-full and buffer-empty conditions, and prevent data loss while maintaining efficient device communication.
4. A high-speed embedded platform contains multiple communication devices competing for shared system resources. Develop pseudocode for a priority-based resource allocation mechanism that assigns communication resources according to device urgency and bandwidth requirements. The algorithm should dynamically update priorities, allocate resources to eligible devices, release resources after transmission, and prevent low-priority devices from being indefinitely delayed.
5. A real-time system must transfer data from several peripheral devices while meeting strict timing constraints. Write pseudocode for a real-time I/O scheduling mechanism that determines when each device should perform data transfers based on deadlines, transfer duration, priority, and available system bandwidth. The algorithm should schedule time-critical transfers first, monitor missed deadlines, and reschedule pending requests to improve overall system responsiveness.

Code of Conduct

I RAGAVIS / 192565027 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: RAGAVIS

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator

Faculty Incharge

Question 1: Nested (Vectored) Interrupt Handling with Priority Pre-emption

(a) Problem Understanding

- Inputs: interrupt request (IRQ id, priority level) arriving from sensors, communication modules, storage devices at unpredictable times.
- Outputs: correctly serviced interrupts in priority order, with the original routine resumed exactly where it left off.
- Constraints: an in-service routine may itself be pre-empted only by a strictly higher priority IRQ; lower/equal priority IRQs must wait in a pending queue; context must never be lost.

(b) Data Structures Used

- `vectorTable[ ]` — maps IRQ id $\rightarrow$ address of its Interrupt Service Routine (ISR).
- `contextStack` — LIFO stack storing {PC, registers, flags, priority} for every pre-empted routine.
- `pendingQueue` — priority queue holding IRQs that arrived but could not pre-empt immediately.
- `currentPriority` — priority level of the routine presently executing (0 = base/no interrupt).

(c) Algorithm — Pseudocode

```
MODULE NestedInterruptHandler
```

```
  GLOBAL currentPriority  $\leftarrow$  0
```

```
  GLOBAL contextStack  $\leftarrow$  EMPTY_STACK
```

```
  GLOBAL pendingQueue  $\leftarrow$  EMPTY_PRIORITY_QUEUE
```

```
ON HARDWARE_INTERRUPT(irqId, irqPriority):
```

```
  IF irqPriority > currentPriority THEN
```

```
    PREEMPT_AND_SERVICE(irqId, irqPriority)
```

```
  ELSE
```

```
    INSERT (irqId, irqPriority) INTO pendingQueue // queued, cannot pre-empt
```

```
  END IF
```

```
FUNCTION PREEMPT_AND_SERVICE(irqId, irqPriority):
```

```
  PUSH {PC, registers, flags, currentPriority} ONTO contextStack // save context
```

```
  MASK all interrupts WITH priority  $\leq$  irqPriority // vectored masking
```

```
  currentPriority  $\leftarrow$  irqPriority
```

```
  vectorAddr  $\leftarrow$  vectorTable[irqId]
```

```
  CALL ISR AT vectorAddr // may itself be pre-empted  $\rightarrow$  recursive nesting
```

```
// ---- ISR finished ----
{PC, registers, flags, currentPriority} ← POP contextStack // restore context
UNMASK interrupts to previous level
RESUME interrupted routine AT restored PC
SERVICE_PENDING()
```

```
FUNCTION SERVICE_PENDING():
  WHILE pendingQueue IS NOT EMPTY DO
    next ← EXTRACT_MAX_PRIORITY(pendingQueue)
    IF next.priority > currentPriority THEN
      PREEMPT_AND_SERVICE(next.irqId, next.priority)
    ELSE
      BREAK // remaining pending IRQs wait for next cycle
    END IF
  END WHILE
END MODULE
```

(d) Control Structures Employed

This solution uses conditional branching (IF/ELSE), pre/post-tested loops (WHILE, REPEAT-UNTIL, LOOP FOREVER), iteration over collections (FOR EACH), modular decomposition (independent FUNCTION/MODULE blocks), and event-driven triggers (ON <event>) — matching the constructs appropriate for interrupt- and I/O-level system programming.

(e) Completeness — Cases Handled

- Higher-priority pre-emption of an already executing ISR (nesting) — handled recursively in PREEMPT_AND_SERVICE.
- Simultaneous/queued lower-priority interrupts — buffered safely in pendingQueue, never lost.
- Context integrity — every pre-emption pushes a full context frame; restoration is symmetric (LIFO), guaranteeing correct resumption even for multiple nested levels.
- Starvation of pending IRQs is bounded because SERVICE_PENDING() is re-evaluated after every ISR return.

Question 2: Scatter–Gather DMA Transfer

(a) Problem Understanding

- Inputs: a linked list of memory descriptors, each holding {sourceAddr, destAddr, length, nextPtr}, without requiring contiguous memory.

- Outputs: all blocks transferred with per-descriptor status (PENDING / IN_PROGRESS / DONE / ERROR) and a single completion/error interrupt to the CPU.
- Constraints: CPU must not be occupied during the transfer (true DMA); transient errors should be retried before aborting.

(b) Data Structures Used

- descriptorList — linked list of descriptors, each with a status field and nextDescriptorPtr.
- DMA control/status registers — SRC_REG, DEST_REG, LEN_REG used to program each burst.
- retryCount — per-descriptor counter bounded by MAX_RETRY.

(c) Algorithm — Pseudocode

```

FUNCTION SCATTER_GATHER_DMA(descriptorList):
  FOR EACH d IN descriptorList DO
    d.status ← PENDING
  END FOR

  current ← descriptorList.HEAD
  WHILE current ≠ NULL DO
    current.status ← IN_PROGRESS
    retryCount ← 0
    REPEAT
      SET DMA_SRC_REG ← current.sourceAddr
      SET DMA_DEST_REG ← current.destAddr
      SET DMA_LEN_REG ← current.length
      START_DMA_BURST() // CPU is free during this burst
      WAIT FOR burstComplete OR burstError // event driven, no polling
      IF burstComplete THEN
        current.status ← DONE
      ELSE
        retryCount ← retryCount + 1
        LOG_ERROR(current)
      END IF
    UNTIL current.status = DONE OR retryCount > MAX_RETRY

    IF current.status ≠ DONE THEN
      current.status ← ERROR
      RAISE_INTERRUPT(DMA_ERROR, current)
      RETURN FAILURE // abort remaining descriptors
    END IF
  
```

```

    current ← current.nextDescriptorPtr
END WHILE

RAISE_INTERRUPT(DMA_COMPLETE) // all descriptors transferred
RETURN SUCCESS
END FUNCTION

```

(d) Control Structures Employed

This solution uses conditional branching (IF/ELSE), pre/post-tested loops (WHILE, REPEAT-UNTIL, LOOP FOREVER), iteration over collections (FOR EACH), modular decomposition (independent FUNCTION/MODULE blocks), and event-driven triggers (ON <event>) — matching the constructs appropriate for interrupt- and I/O-level system programming.

(e) Completeness — Cases Handled

- Non-contiguous memory handled naturally by traversing the descriptor linked list rather than one flat address range.
- Transient burst errors are retried up to MAX_RETRY before the whole transfer is aborted.
- Descriptor status is updated at every stage, giving the CPU full visibility for diagnostics.
- A single completion interrupt (rather than one per block) keeps CPU overhead minimal, satisfying the 'no continuous CPU occupation' requirement.

Question 3: I/O Buffering Mechanism (Bounded Circular Buffer)

(a) Problem Understanding

- Inputs: byte/record stream arriving from a device (producer) at a variable rate, and consumption requests from the CPU/process (consumer).
- Outputs: no data loss on producer side, no invalid reads on consumer side, and flow-control signals when occupancy is extreme.
- Constraints: fixed buffer capacity (bufferSize); must handle producer faster than consumer and vice-versa.

(b) Data Structures Used

- buffer[0..bufferSize-1] — circular array.
- head, tail — read/write indices (mod bufferSize).
- count — current occupancy; HIGH_WATERMARK / LOW_WATERMARK — thresholds for flow control.

(c) Algorithm — Pseudocode

```

FUNCTION INIT_BUFFER(size):
    bufferSize ← size ; buffer ← NEW ARRAY[size]
    head ← 0 ; tail ← 0 ; count ← 0
END FUNCTION

FUNCTION WRITE_TO_BUFFER(dataItem): // called by device/producer
    IF count = bufferSize THEN
        SIGNAL(BUFFER_FULL)
        APPLY_BACKPRESSURE(device) // pause / drop-oldest per policy, avoid
        loss
        RETURN BUSY
    END IF
    buffer[tail] ← dataItem
    tail ← (tail + 1) MOD bufferSize
    count ← count + 1
    RETURN OK
END FUNCTION

FUNCTION READ_FROM_BUFFER(): // called by CPU/consumer
    IF count = 0 THEN
        SIGNAL(BUFFER_EMPTY)
        RETURN NO_DATA
    END IF
    dataItem ← buffer[head]
    head ← (head + 1) MOD bufferSize
    count ← count - 1
    RETURN dataItem
END FUNCTION

FUNCTION MONITOR_BUFFER(): // continuous background task
    LOOP FOREVER DO
        occupancy ← (count / bufferSize) × 100
        IF occupancy ≥ HIGH_WATERMARK THEN
            THROTTLE(device) // slow producer / trigger secondary spill buffer
        ELSE IF occupancy ≤ LOW_WATERMARK THEN
            RESUME_NORMAL_RATE(device)
        END IF
    END LOOP
END FUNCTION

```

(d) Control Structures Employed

This solution uses conditional branching (IF/ELSE), pre/post-tested loops (WHILE, REPEAT-UNTIL, LOOP FOREVER), iteration over collections (FOR EACH),

modular decomposition (independent FUNCTION/MODULE blocks), and event-driven triggers (ON <event>) — matching the constructs appropriate for interrupt- and I/O-level system programming.

(e) Completeness — Cases Handled

- Buffer-full: producer is either blocked, throttled, or overflow is spilled per policy — never silently overwritten, so no data loss.
- Buffer-empty: consumer receives an explicit NO_DATA signal instead of reading garbage.
- Watermark-based monitoring smooths short bursts before they become full/empty events.
- Circular indexing reuses freed slots automatically — no explicit release step required, keeping allocation $O(1)$.

Question 4: Priority-Based Resource Allocation with Ageing (Bus Arbitration)

(a) Problem Understanding

- Inputs: a set of competing device requests, each with {basePriority, bandwidthNeed, waitTime}.
- Outputs: bandwidth/bus grants to eligible devices and timely release once transmission completes.
- Constraints: total allocation must never exceed availableBandwidth; low-priority devices must not be delayed indefinitely.

(b) Data Structures Used

- requestQueue — set of pending device requests.
- availableBandwidth — shared resource pool.
- AGING_FACTOR — bonus added to priority per unit wait time (starvation prevention).

(c) Algorithm — Pseudocode

```

FUNCTION RESOURCE_ALLOCATOR():
  LOOP FOREVER DO
    UPDATE_PRIORITIES()
    sortedQueue ← SORT(requestQueue, BY effectivePriority DESCENDING)
    FOR EACH req IN sortedQueue DO
      IF req.bandwidthNeed ≤ availableBandwidth THEN
        GRANT resource TO req.device
        availableBandwidth ← availableBandwidth - req.bandwidthNeed
        req.status ← ALLOCATED
        START_TRANSMISSION_TIMER(req.device)
  
```

```

    REMOVE req FROM requestQueue
ELSE
    req.waitTime ← req.waitTime + 1 // will raise effectivePriority next
    cycle
END IF
END FOR
END LOOP
END FUNCTION

FUNCTION UPDATE_PRIORITIES():
    FOR EACH req IN requestQueue DO
        req.effectivePriority ← req.basePriority + (req.waitTime × AGING_FACTOR)
    END FOR
END FUNCTION

ON TRANSMISSION_COMPLETE(device): // interrupt-driven release
    availableBandwidth ← availableBandwidth + device.allocatedBandwidth
    STOP_TRANSMISSION_TIMER(device)
    LOG(RELEASE, device)

```

(d) Control Structures Employed

This solution uses conditional branching (IF/ELSE), pre/post-tested loops (WHILE, REPEAT-UNTIL, LOOP FOREVER), iteration over collections (FOR EACH), modular decomposition (independent FUNCTION/MODULE blocks), and event-driven triggers (ON <event>) — matching the constructs appropriate for interrupt- and I/O-level system programming.

(e) Completeness — Cases Handled

- Bandwidth is never over-committed — a grant only occurs if req.bandwidthNeed fits availableBandwidth.
- Resources are released promptly on the TRANSMISSION_COMPLETE event, making them available the very next cycle.
- Ageing guarantees an upper bound on wait time — effectivePriority strictly increases each cycle a request is skipped, so it eventually outranks all others (starvation prevented).
- Dynamic re-sorting every cycle reflects both urgency changes and newly arrived requests.

Question 5: Real-Time I/O Scheduling (Earliest-Deadline-First with Reschedule)

(a) Problem Understanding

- Inputs: device requests each with {arrivalTime, duration, deadline, priority, bandwidthNeed}.
- Outputs: a per-tick schedule of transfers that meets deadlines whenever feasible, plus logged/rescheduled misses.
- Constraints: total bandwidth used per tick $\leq$ systemBandwidth; requests that cannot meet their deadline must be detected, not silently dropped.

(b) Data Structures Used

- deviceRequests — pool of all requests (pending / in-progress / missed).
- eligible — subset that has arrived and is still pending, sorted by deadline (EDF).
- waitQueue — requests deferred to the next tick due to bandwidth limits.

(c) Algorithm — Pseudocode

```

FUNCTION REALTIME_IO_SCHEDULER(deviceRequests):
  LOOP AT EVERY SCHEDULING TICK DO
    eligible ← FILTER(deviceRequests, arrivalTime ≤ currentTime AND status =
    PENDING)
    SORT eligible BY deadline ASCENDING, TIE-BREAK BY priority
    DESCENDING // EDF
    usedBandwidth ← 0

    FOR EACH req IN eligible DO
      IF currentTime + req.duration > req.deadline THEN
        req.status ← MISSED_DEADLINE
        LOG_MISSED(req)
        RESCHEDULE(req)
        CONTINUE
      END IF

      IF usedBandwidth + req.bandwidthNeed ≤ systemBandwidth THEN
        SCHEDULE_TRANSFER(req, currentTime)
        usedBandwidth ← usedBandwidth + req.bandwidthNeed
        req.status ← IN_PROGRESS
      ELSE
        ADD req TO waitQueue // deferred to next tick, deadline re-checked then
      END IF
    END FOR

    currentTime ← currentTime + tickInterval
  END LOOP
END FUNCTION

```

```

FUNCTION RESCHEDULE(req):
    req.deadline ← currentTime + req.gracePeriod
    req.priority ← req.priority + PENALTY_BOOST // raise urgency after a miss
    req.status ← PENDING
    ADD req TO deviceRequests
END FUNCTION

FUNCTION MONITOR_DEADLINES(): // background watchdog
    LOOP FOREVER DO
        FOR EACH req IN deviceRequests DO
            IF req.status = PENDING AND currentTime > req.deadline THEN
                RESCHEDULE(req)
            END IF
        END FOR
    END LOOP
END FUNCTION

```

(d) Control Structures Employed

This solution uses conditional branching (IF/ELSE), pre/post-tested loops (WHILE, REPEAT-UNTIL, LOOP FOREVER), iteration over collections (FOR EACH), modular decomposition (independent FUNCTION/MODULE blocks), and event-driven triggers (ON <event>) — matching the constructs appropriate for interrupt- and I/O-level system programming.

(e) Completeness — Cases Handled

- Time-critical transfers are always scheduled first via EDF sorting each tick.
- Bandwidth-limited ticks defer (not drop) excess requests through waitQueue, re-evaluated next tick.
- MONITOR_DEADLINES independently catches any miss RESULT loop did not see, so nothing is silently lost.
- RESCHEDULE raises both deadline (grace period) and priority, so a missed request is more likely to succeed on retry — improving overall responsiveness.